Name: VNguyen

Date: March 8th, 2026

Course: IT FDN 130 GP

GitHub: https://github.com/Victornguyen527/DBFoundations-Module07

Assignment 01: Creating and Using Databases

Introduction

In this writeup I will explain my understanding of when to use UDFs, and the difference between Scalar, Inline, and Multi-Statement Functions.

SQL UDF

I would use UDFs whenever I want to simplify complex queries or reuse code. For example, in Assignment 7, we used an inline function to simplify retrieving tables using the KPI value.

```
create function fProductInventoriesWithPreviousMonthCountsWithKPIs (@KPI INT)
    returns table as return
    select top 1000
    ProductName,
    InventoryDate,
    InventoryCount,
    PreviousMonthCount,
    CountVsPreviousCountKPI
    from vProductInventoriesWithPreviousMonthCountsWithKPIs
    where CountVsPreviousCountKPI = @KPI
    group by ProductName, InventoryDate, InventoryCount, PreviousMonthCount, CountVsPreviousCountKPI
    order by ProductName, Month(InventoryDate)
go
```

With the inline function in place, we can retrieve tables of product rows just by using a SELECT statement on the function name and the KPI value we want to retrieve.

```
select * from fProductInventoriesWithPreviousMonthCountsWithKPIs(1)
select * from fProductInventoriesWithPreviousMonthCountsWithKPIs(0)
select * from fProductInventoriesWithPreviousMonthCountsWithKPIs(-1)
```

Creating a function allows us to reuse all the code except the WHERE clause. Without the function, we would have to type out the whole SELECT statement and change the WHERE clause to search for a specific KPI value each time we want to look up a different KPI value, like so.

```sql
select
    ProductName,
    InventoryDate,
    InventoryCount,
    PreviousMonthCount,
    CountVsPreviousCountKPI
    from vProductInventoriesWithPreviousMonthCountsWithKPIs
    where CountVsPreviousCountKPI = '1'
    order by ProductName, Month(InventoryDate)
```

```sql
select
    ProductName,
    InventoryDate,
    InventoryCount,
    PreviousMonthCount,
    CountVsPreviousCountKPI
    from vProductInventoriesWithPreviousMonthCountsWithKPIs
    where CountVsPreviousCountKPI = '0'
    order by ProductName, Month(InventoryDate)
```

```sql
select
    ProductName,
    InventoryDate,
    InventoryCount,
    PreviousMonthCount,
    CountVsPreviousCountKPI
    from vProductInventoriesWithPreviousMonthCountsWithKPIs
    where CountVsPreviousCountKPI = '-1'
    order by ProductName, Month(InventoryDate)
```

Scalar, Inline, and Multi-Statement Functions

Scalar functions take parameters, perform operations, and return a single value based on the input parameters. They can take multiple parameters but can only return one result.

Inline functions are similar to scalar functions because they can accept multiple parameters, perform operations, and return data. However, inline functions can return more than one value, and they return the data as a table, unlike scalar functions. Notably, inline functions can include only a single SELECT statement.

Multi-statement functions are like inline functions and return results as a table with multiple rows and columns, but they also allow multiple SQL statements. Multi-statement functions

require that the table's structure, such as column names and data types, be explicitly defined. They also need to include BEGIN and END blocks.

Summary

UDFs should be used to simplify complex queries or to reuse code. Scalar functions take parameters, perform operations, and return a single value. Inline functions also take parameters, but they return multiple results in the form of a table. Inline functions can include only a single SELECT statement. Multi-statement functions can include multiple SELECT statements. A multi-statement function must include BEGIN and END blocks, and the table's structure, such as column names and data types, must be explicitly defined.